

University of Engineering and Technology, Lahore



Mr_Lab_Report_9

Submitted by: Nayab Noor (2022_MC_27)

1. Objectives

The primary objectives of this laboratory session are:

- Subscribe to and process live image data from the TurtleBot3 camera using the ROS 2 `/camera/image_raw` topic.
- Apply OpenCV-based color segmentation in the HSV color space to detect and isolate colored objects in the environment.
- Compute the centroid of a detected colored object and measure the horizontal error relative to the image center.
- Implement a proportional (P) controller to generate angular velocity commands that align the robot with the detected target.
- Develop approach behavior so the robot moves forward when aligned and stops at close range using area-based thresholding.
- Experimentally tune key control parameters including the proportional gain (k_p) and HSV threshold values.
- Extend the system to support multi-color detection and assign distinct robot behaviors to each color class.

2. Introduction

Vision-based robot control is a cornerstone of autonomous mobile robotics. By leveraging onboard cameras and image processing libraries, a robot can perceive its surroundings, detect objects of interest, and make real-time navigational decisions without the need for explicit waypoints or pre-mapped environments. This lab explores the integration of ROS 2, OpenCV, and feedback control on the TurtleBot3 Waffle platform inside the Gazebo simulation environment.

The TurtleBot3 Waffle is equipped with a forward-facing camera that publishes frames to the `/camera/image_raw` topic. Using the `cv_bridge` library, these ROS 2 Image messages are converted into OpenCV-compatible NumPy arrays. The frames are then processed in the Hue-Saturation-Value (HSV) color space, which is significantly more robust to lighting variation than the default BGR representation.

A binary mask is generated by thresholding the HSV image within a predefined color range. Morphological operations (erosion and dilation) clean up the mask by removing noise and filling gaps. The spatial moments of the resulting mask are computed to find the centroid of the detected object. The horizontal offset of this centroid from the image center constitutes the tracking error.

A proportional controller maps this error directly to an angular velocity command published on the `/cmd_vel` topic. When the error falls below a defined threshold (the robot is aligned), a forward velocity is also commanded so the robot approaches the target. Approach stops when the contour area exceeds a preset stop threshold, indicating proximity. A search mode rotates the robot slowly when no target is visible. Advanced exercises extend these concepts to true 360-degree odometry-tracked search and multi-color priority-based behavior selection.

3. Code Explanation

3.1 Task 1: Camera Subscriber and Display (task_1.py)

Task 1 establishes the foundational ROS 2 node that subscribes to the camera topic and displays live video using OpenCV. The node class `CameraViewer` inherits from `rclpy.Node`. Inside the constructor, a subscription is created on the `/camera/image_raw` topic using the `Image` message type with a queue depth of 10. A `CvBridge` object converts ROS `Image` messages to OpenCV arrays.

In the `image_callback` method, each incoming frame is decoded to BGR8 format using `imgmsg_to_cv2`. The `cv2.imshow` call renders the frame in a named window, and `cv2.waitKey(1)` allows OpenCV to process GUI events. This simple pipeline verifies that the camera topic is active and accessible before any image processing is introduced.

Key concept: The `cv_bridge` library bridges the gap between ROS 2 `sensor_msgs/Image` and OpenCV's `Mat` (NumPy array in Python), handling encoding conversion transparently.

3.2 Task 2: HSV Segmentation (task_2.py)

Task 2 extends the camera viewer by adding color-based segmentation. After converting the frame to HSV using `cv2.cvtColor`, a lower and upper HSV boundary for the color green is defined. The `cv2.inRange` function produces a binary mask where white pixels correspond to pixels within the color range.

Two morphological operations follow: `MORPH_OPEN` (erosion then dilation) eliminates small noise specks, and `MORPH_DILATE` enlarges the remaining regions to fill any internal gaps. Both the original color frame and the binary mask are displayed side-by-side, allowing visual inspection of segmentation quality. Students observe how adjusting the hue, saturation, and value bounds directly controls which pixels are retained.

HSV range chosen: Hue 35-85 (green band), Saturation 80-255 (vivid colors), Value 80-255 (adequate brightness). This range was selected to work reliably under the uniform lighting of the Gazebo simulation.

3.3 Task 3: Centroid Detection (task_3.py)

Task 3 introduces object localisation through image moments. After generating and cleaning the binary mask, `cv2.moments(mask)` computes the zeroth moment (total white pixel area, `m00`) and the first moments in x and y (`m10`, `m01`). The centroid coordinates are derived as:

```
cx = int(M['m10'] / M['m00'])
cy = int(M['m01'] / M['m00'])
```

The horizontal error is defined as the signed difference between the detected centroid and the horizontal center of the image (`screen_center_x = width // 2`). A positive error means the object is to the left of center; a negative error means it is to the right. This sign convention determines the direction of the corrective angular rotation.

Visualisation overlays are drawn on the frame: a cross-hair marker at the centroid, and text annotations displaying the centroid coordinates and error value. If no green pixels are found (`m00 == 0`), a 'Target Lost' message is displayed.

3.4 Task 4: Proportional Alignment (`task_4.py`)

Task 4 introduces closed-loop control. A velocity publisher is added on `/cmd_vel`, publishing `geometry_msgs/Twist` messages. The proportional gain `kp` is set to 0.0015 and the stop threshold is 12 pixels.

The control law is straightforward: the angular velocity command is computed as the product of `kp` and the error. If the absolute error is less than or equal to the `stop_threshold` (12 pixels), the angular command is set to zero and the robot holds position. Otherwise, the angular velocity is clipped to a maximum of 0.3 rad/s to prevent aggressive motion:

```
angular_velocity = self.kp * error
cmd.angular.z = np.clip(angular_velocity, -max_speed, max_speed)
```

When no object is detected (mask area below 200 pixels), the robot rotates at 0.2 rad/s to search for the target. A rectangle is drawn on the display marking the 12-pixel stop zone around the image center, providing clear visual feedback of the alignment window.

3.5 Task 5: Full Approach and Stop (`task_5.py`)

Task 5 completes the full perception-to-navigation pipeline. The node now tracks the green object using contour-based detection rather than raw mask moments, which is more robust since it isolates the largest coherent region rather than averaging all green pixels in the image.

After finding all contours and filtering by `min_area` (500 pixels), the largest contour is selected. Its area is used for proximity estimation: if the area exceeds `stop_area` (550,000 pixels), the robot has arrived and both linear and angular velocities are set to zero. If the horizontal error exceeds `center_threshold` (40 pixels), the robot rotates in place to align. Once aligned, the robot advances at `approach_speed` (0.15 m/s) while continuing to apply a small angular correction. If no object is detected, the robot enters a slow spin search at `search_speed` (0.20 rad/s):

```
if area >= self.stop_area:
    twist.linear.x = 0.0; twist.angular.z = 0.0    # STOPPED
elif abs(error) > self.center_threshold:
    twist.linear.x = 0.0                          # ALIGNING
    twist.angular.z = self.kp * error
```

```

else:
    twist.linear.x = self.approach_speed          # APPROACHING
    twist.angular.z = self.kp * error

```

3.6 Exercise 1: Blue Object Detection (excercise_1.py)

Exercise 1 adapts the Task 5 pipeline to detect a blue object instead of green. The only modification is the HSV boundary values:

```

self.lower_color = np.array([100, 150, 50])
self.upper_color = np.array([140, 255, 255])

```

Blue occupies hue values in the range 100-140 in the OpenCV HSV convention (0-179). Higher saturation and value minima (150, 50) are used to reject grey or near-white surfaces that might share some hue overlap. All control parameters (kp, center_threshold, approach_speed, stop_area) remain unchanged, demonstrating the modularity of the design.

3.7 Exercise 2: Proportional Gain Tuning (excercise_2.py)

Exercise 2 investigates the effect of changing the proportional gain kp from 0.001 (Exercise 1) to 0.0005. A smaller kp produces slower but smoother angular corrections. The robot takes longer to align but is less prone to overshoot and oscillation around the center. All other parameters are held constant so the kp effect can be isolated.

3.8 Exercise 3: Odometry-Tracked 360-Degree Search (excercise_3.py)

Exercise 3 improves the search behavior by replacing the naive indefinite spin with a true 360-degree scan. A subscription to /odom provides the robot's quaternion orientation. The yaw angle is extracted from the quaternion using:

```

siny_cosp = 2 * (q.w * q.z + q.x * q.y)
cosy_cosp = 1 - 2 * (q.y * q.y + q.z * q.z)
yaw = math.atan2(siny_cosp, cosy_cosp)

```

On each callback, the angular delta between the current and previous yaw is computed and accumulated after normalising wrap-around to the range $[-\pi, \pi]$. Once the accumulated rotation exceeds 6.2 radians (approximately 2π), rotation stops. This guarantees the robot performs exactly one full revolution before concluding no target is present, rather than spinning indefinitely.

3.9 Exercise 4: Multi-Color Priority Behavior (excercise_4.py)

Exercise 4 is the most complex task. Three color classes are tracked simultaneously, each triggering a different robot behavior, evaluated in priority order:

- Red (highest priority): Robot moves backward at -0.15 m/s while keeping the red centroid horizontally centered. This models obstacle avoidance.
- Blue (medium priority): Robot rotates in place to center the blue object, then stops completely when centered within the threshold. This models precision docking.
- Green (lowest priority): Robot aligns and approaches as in Task 5, stopping at close range. This is the navigation goal.
- No target found: 360-degree odometry-tracked scan (same mechanism as Exercise 3).

Red detection uses dual HSV bounds to handle the hue wrap-around at 0/180 degrees: one range covers hue 0-10 and a second covers hue 170-180. The two masks are combined with `cv2.bitwise_or` before contour analysis. A helper method `analyze_mask` encapsulates the morphological cleanup and moment computation, keeping the main callback concise. A combined mask of all three detections is shown in a single display window.

4. Observations on Controller Tuning and Color Segmentation

4.1 Proportional Gain (kp) Tuning

The proportional gain `kp` directly scales the angular velocity output. Its effect was studied across three regimes:

kp Value	Behavior	Observation
0.002 (High)	Oscillatory	Robot overshoots center repeatedly. Angular velocity peaks cause hunting behavior.
0.001 (Nominal)	Smooth	Clean alignment with minimal overshoot. Chosen as default for Tasks 1-5 and Exercises 1, 3, and 4.
0.0005 (Low)	Sluggish	Alignment takes longer but is very smooth. Used in Exercise 2 to demonstrate reduced oscillation at cost of speed.

The `center_threshold` parameter was equally important. A threshold that is too tight (e.g., 5 pixels) causes the robot to oscillate at the boundary; too large (e.g., 100 pixels) means the robot approaches while still significantly misaligned. The value of 40 pixels proved well-balanced for the simulation camera resolution.

4.2 Color Segmentation Quality

Segmentation quality was evaluated by varying the HSV bounds and observing mask coverage and noise levels.

- Green detection: The hue range 35-85 reliably captured the Gazebo green box while rejecting yellow and cyan surfaces. Raising the saturation minimum to 80 excluded grey concrete textures. Without the MORPH_OPEN step, scattered single pixels caused spurious small contours, resulting in erratic centroid jumps.
- Blue detection: Hue range 100-140 with saturation minimum 150 was necessary to differentiate the blue target from the sky (which also has blue tones at lower saturation in some Gazebo scenes). A tighter hue range would improve robustness in complex outdoor scenes.
- Red detection: Red wraps around the HSV hue axis (at both 0 and 180). Combining two separate inRange masks and merging with bitwise_or is the standard solution. Omitting either range causes partial detection under different lighting angles.
- Morphological tuning: A 5x5 kernel with 1 erosion and 2 dilation iterations struck the best balance between noise removal and object preservation. Larger kernels caused small or distant objects to disappear entirely.

4.3 Area-Based Stop Threshold

The stop_area threshold of 550,000 pixels was set empirically by placing the robot at the desired stopping distance from the target in the simulation and recording the contour area at that position. This value is resolution-dependent and would need recalibration for different camera resolutions or stop distances. The min_area filter of 500 pixels effectively rejected reflection glints and boundary artefacts.

4.4 360-Degree Search Reliability

The naive timed search (implicit in the basic search_speed spin without odometry) was unreliable because ROS 2 callback timing is not deterministic. The odometry-based approach accumulates true angular displacement, ensuring exactly one full revolution is performed regardless of callback frequency or motor latency. The threshold of 6.2 radians (slightly less than 2π to account for floating-point accumulation) worked reliably in all test runs.

5. Exercise Questions and Answers

Exercise 1: How does changing the HSV threshold affect detection quality?

Answer: Changing the HSV threshold values selects a different portion of the color spectrum for the binary mask. For blue detection, the hue range is shifted from 35-85 (green) to 100-140 (blue). The saturation minimum was increased from 80 to 150 because blue hues appear at moderate

saturation in sky backgrounds in the Gazebo world, so a higher saturation cutoff prevents false positives. The value (brightness) minimum remains at 50, as the blue box is well-illuminated.

Effect on segmentation: A wider hue range detects more blue pixels, including shadows and reflections, but increases false positives. A narrower range is more precise but may fragment detection when the object is viewed at an angle (hue shift due to specular highlights). Lighting changes in the simulation do not significantly affect Gazebo's default lighting but would be a major concern in real-world deployment, where adaptive thresholding or machine-learning-based segmentation would be preferred.

Exercise 2: What is the effect of tuning the proportional gain (kp)?

Answer: The proportional gain k_p scales the control response. The angular velocity command is: $\omega = k_p * \text{error}$. A large k_p (e.g., 0.002) generates large angular velocities for even small errors, causing the robot to overshoot the target center and oscillate back and forth. This is the proportional control equivalent of instability. A small k_p (e.g., 0.0005) produces gentle corrections but slows convergence significantly; the robot may not align before the target moves.

The nominal value of $k_p = 0.001$ was found to provide critically-damped-like behavior in the Gazebo simulation: the robot aligns within 2-3 seconds from a 200-pixel error without visible oscillation. Adding a derivative term (D-control) would further suppress overshoot, but was beyond the scope of this lab. For very small errors (near the `center_threshold`), the angular velocity becomes very small and may not overcome motor static friction in real hardware, making the threshold important for preventing stalling.

Exercise 3: How does odometry-based search improve behavior?

Answer: The basic search (constant angular velocity until an object is detected) does not know how far the robot has rotated. If no object is in the scene, the robot spins indefinitely, which is undesirable. Odometry-based tracking accumulates the true angular displacement by integrating yaw deltas between consecutive odometry callbacks. Once the accumulated rotation reaches 2π radians (approximately 6.28, but 6.2 used as threshold to ensure completion despite floating-point rounding), the robot knows it has completed a full 360-degree scan and stops.

This approach is robust to non-uniform callback timing because it measures rotation in angle units rather than time. The wrap-around normalization (subtracting 2π if $\text{diff} > \pi$) correctly handles the discontinuity at the ± 180 -degree boundary in the `atan2` output. When an object is found during the scan, the searching flag is reset and accumulated angle is cleared, ready for the next search session.

Exercise 4: How are multiple colors handled with different behaviors?

Answer: Exercise 4 implements a behavior priority tree. All three color masks (red, blue, green) are computed every frame. The decision logic evaluates them in strict priority order using if-elif-else:

- Red (highest): Immediate backward motion to create distance from the obstacle. The robot also steers to keep red centered, preventing it from losing sight of the hazard.
- Blue (medium): Precision alignment by rotating in place until the centroid is within the threshold, then stopping. No forward motion is commanded for blue, unlike green.
- Green (lowest): Standard approach behavior: align, move forward, stop at close range.
- None found: Odometry-based 360-degree scan.

The priority ensures safe behavior: if both red and green are visible simultaneously, the robot backs away from red rather than approaching green. The helper method `analyze_mask` prevents code duplication across the three color branches. Red required dual HSV bounds due to the circular hue axis. The combined mask display (`bitwise_or` of all three) lets the operator visually verify which colors are being detected simultaneously.

6. Perception-to-Control Pipeline

The complete data flow implemented in this lab is summarised below:

```
/camera/image_raw (sensor_msgs/Image)
|
v
cv_bridge: imgmsg_to_cv2 --> BGR Frame
|
v
cv2.cvtColor(BGR -> HSV)
|
v
cv2.inRange(HSV, lower, upper) --> Binary Mask
|
v
Morphological Ops (erode + dilate) --> Clean Mask
|
v
cv2.findContours / cv2.moments --> Centroid (cx, cy)
|
v
Error = img_center_x - cx
|
v
Proportional Control: omega = kp * error
|
v
/cmd_vel (geometry_msgs/Twist)
```

8. Conclusion

This lab provided comprehensive hands-on experience in vision-based perception and control using ROS 2, OpenCV, and the TurtleBot3 Waffle in Gazebo simulation. The following key learning outcomes were achieved:

- **ROS 2 Image Pipeline:** Students successfully subscribed to the `/camera/image_raw` topic, converted Image messages to OpenCV arrays using `cv_bridge`, and displayed real-time video. The publisher-subscriber pattern and message types (`sensor_msgs/Image`, `geometry_msgs/Twist`, `nav_msgs/Odometry`) were used in practice.
- **HSV Color Segmentation:** The advantages of the HSV color space over BGR for color-based detection were demonstrated. The three-parameter threshold (H, S, V) provides independent control over color identity, vividness, and brightness, making it more robust than simple RGB thresholding.
- **Centroid Computation:** Image moments provided a computationally efficient method to find the center of mass of a binary region. The horizontal offset from the image center served as a meaningful error signal for closed-loop control.
- **Proportional Control:** The P-controller effectively translated a pixel-domain error into an angular velocity command. The tuning exercise (Exercise 2) gave concrete insight into the trade-off between response speed and oscillation, a fundamental concept in control theory.
- **Odometry Integration:** Using quaternion-to-Euler conversion and yaw accumulation, the robot performed reliable 360-degree scans rather than indefinite spinning, demonstrating the importance of sensor feedback in search strategies.
- **Multi-Color Behavior:** Exercise 4 demonstrated how a priority tree can implement complex autonomous behaviors from simple building blocks, a design principle used widely in autonomous systems and behavior-based robotics.

The perception-to-control pipeline demonstrated in this lab, from raw sensor data through feature extraction to closed-loop motion, is a microcosm of the broader autonomy stack used in real mobile robot systems. Future extensions could include replacing the proportional controller with a PID controller to reduce steady-state error, or substituting HSV thresholding with a neural network detector for improved performance in uncontrolled lighting.